

Gait Analysis Using Python

1. Introduction

Motion capture and force plate analysis are widely used to quantify human gait and evaluate lower-limb biomechanics. While commercial software provides extensive post-processing capabilities, custom programming environments allow greater flexibility, automation and reproducibility.

This project revisits a gait analysis originally performed during undergraduate studies [1], where the author acted as the test participant. Rather than repeating the original biomechanical interpretation, the objective is to reproduce the complete processing pipeline using Python while validating the results against the original analysis and extending it with additional gait parameters including walking speed and pelvic vertical excursion.

2. Methodology

2.1 Original Dataset

The dataset analysed in this project originated from a biomechanics laboratory assignment completed during undergraduate studies [1], in which the author acted as the test subject. The original experiment involved three-dimensional motion capture and force plate analysis during level walking. The original assignment analysed the data using biomechanical software (MOKKA & VICON), while the present project reprocessed the exported raw data using Python.

2.2 Experimental Setup

The gait trial was recorded using a Vicon motion capture system and AMTI force plates. Reflective markers were attached to anatomical landmarks on the lower limbs and pelvis, allowing three-dimensional marker trajectories to be recorded during walking. Force plates were used to measure the ground reaction forces generated during foot contact. [1]

Parameter	Description
Test subject	Author
Task	Level walking
Motion capture system	Vicon
Force measurement	AMTI force plates
Data analysed	One complete left gait cycle
Reference cycle	Left heel strike to next left heel strike

2.3 Gait Cycle Selection

A complete gait cycle was selected between two consecutive left heel strikes. The cycle began at left heel strike and ended at the following left heel strike of the same limb. Left toe-off and right heel strike were also identified to divide the gait cycle into the relevant stance and swing phases.

The selected cycle was used for all subsequent force, kinematic and kinetic analyses. Data were normalized to 0–100% of the gait cycle to allow comparison with theoretical gait profiles reported in the literature.

2.4 Data Processing

The exported Excel data were imported into Python using the Pandas library. Relevant columns were extracted for time, ground reaction forces, joint angles, joint moments and pelvis marker trajectories. Non-numeric values were converted or removed where necessary to ensure clean numerical processing.

Processing steps included:

- extraction of force plate and marker data,
- time normalisation to 0–100% gait cycle,
- body-weight normalisation of ground reaction forces,
- conversion of joint moments to appropriate units,
- calculation of maximum, minimum and range-of-motion values,
- generation of final graphs using Matplotlib.

2.5 Additional Analyses

In addition to reproducing the original gait analysis, two further calculations were performed using Python.

Walking speed was estimated from the forward displacement of the pelvis over one complete gait cycle. Pelvic vertical oscillation was estimated by averaging the vertical positions of the LASI, RASI, LPSI and RPSI markers. These additional analyses were used to determine whether the subject's gait was unusually fast or excessively “bouncy”.

3. Python Workflow

3.1 Software Environment

Python was used as the main processing environment because it allows reproducible and flexible analysis of biomechanical data. The workflow was developed in Visual Studio Code and relied mainly on Pandas and Matplotlib.

Software / Library	Purpose
Python	Main programming environment
Pandas	Excel import and data manipulation
Matplotlib	Plot generation
OpenPyXL	Reading Excel files
Visual Studio Code	Script development

3.2 Workflow Overview

The workflow followed a structured processing pipeline:

Raw Excel data → Python import using Pandas → Variable extraction → Time and force normalization → Peak and range calculations → Plot generation using Matplotlib → Biomechanical interpretation

This workflow allowed the original gait-analysis results to be reproduced while also enabling additional calculations not included in the first assignment.

3.3 Example Processing Step

The following simplified example shows the approach used to import, normalise and plot the vertical ground reaction force:

```
df = pd.read_excel("Walk_002.xlsx", header=None)

time = pd.to_numeric(df.iloc[126:238, 0], errors="coerce")

force_z = pd.to_numeric(df.iloc[126:238, 6], errors="coerce")

time_norm = ((time - time.iloc[0]) / (time.iloc[-1] - time.iloc[0])) * 100

body_weight = body_mass * 9.81

force_bw_z = force_z / body_weight

plt.plot(time_norm, force_bw_z)
```

```
plt.xlabel("Gait Cycle (%)")  
plt.ylabel("Vertical GRF (BW)")  
plt.show()
```

The same processing logic was applied to the joint angles, joint moments, pelvic displacement and walking speed calculations.

3.4 Advantages of the Python Workflow

Compared with manual post-processing, Python provides a more reproducible workflow. Once the scripts were developed, the same processing steps could be applied consistently to different variables with minimal manual intervention. This reduced repetition, improved consistency between graphs and allowed automatic calculation of key gait parameters such as peak forces, range of motion, walking speed and pelvic vertical excursion.

3.5 Validation of the Workflow

The Python-generated graphs were compared with the outputs from the original assignment. The ground reaction force profiles, joint angle curves and joint moment curves closely reproduced the original biomechanical results. Minor numerical or visual differences may be attributed to differences in graphing, interpolation, normalisation and data extraction methods rather than changes in the underlying gait data.

This confirmed that the Python workflow was suitable for reproducing and extending the original gait analysis.

4. Gait Events Timeline and Parameters

The gait trial was recorded using a force platform and motion capture system. The original marker trajectories and force plate data were exported to Excel and processed using Python.

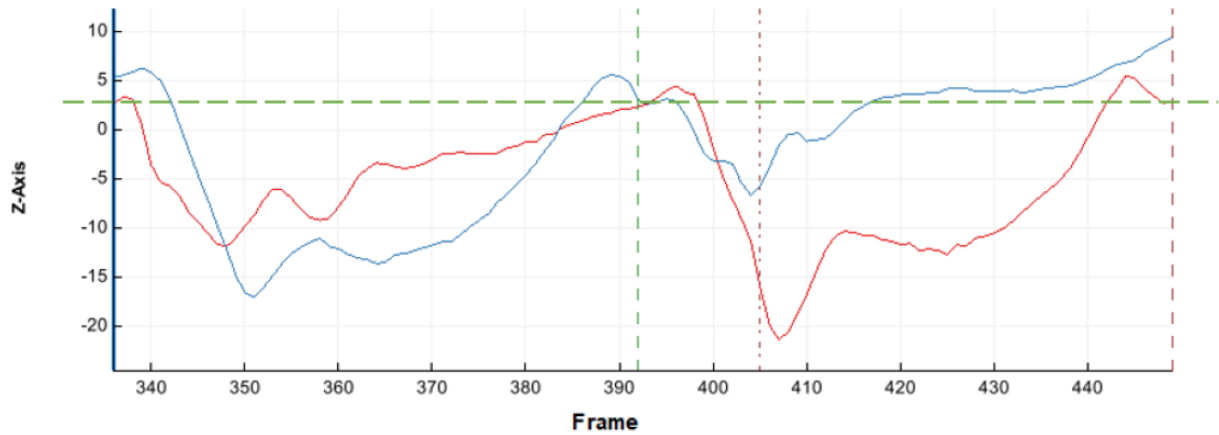


Figure 1 - Complete Gait Cycle timeline from Mokka software obtained during frames 336 to 449 with the red and blue graphs indicating the left and right ankles respectively [1]

The following gait events were identified:

Event	Frame*
Left Heel Strike	335
Left Toe Off	403
Next Left Heel Strike	448

*Each frame is 0.01s

The gait cycle was selected between two consecutive left heel strikes (frames 336–448). Heel strike and toe-off events were identified from the marker trajectory data shown in Figure 1. The resulting gait cycle duration was 1.13 s and was subsequently normalised to 0–100% gait cycle for all kinematic and kinetic analyses.

Python calculated temporal parameters:

Parameter	Value
Gait cycle time	1.13 s
Stance time	0.68 s
Swing time	0.45 s
Stance phase	60.2 %
Swing phase	39.8 %
Cadence	106 steps/min

The stance and swing percentages closely matched values reported for healthy adult walking (approximately 60% stance and 40% swing). [2,3]

5. Ground Reaction Forces

The combined ground reaction forces (GRFz, GRFy & GRFx) were plotted during the stance phase from heel strike to toe off (0% to 60% gait). At first glance the shape is identical for all plots. Noticeable discrepancies are that the amplitudes of all the ground reaction components of the test subject are greater than the reference plot. Respective curves are colour coded:

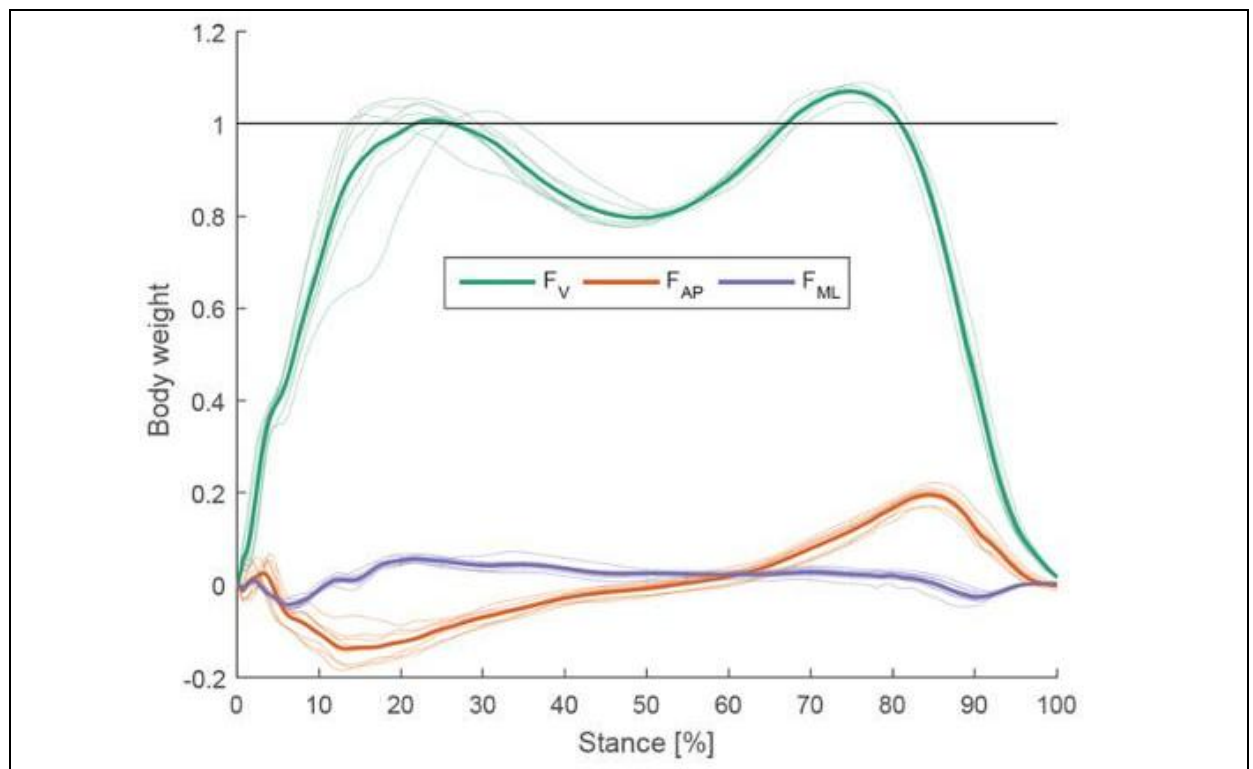


Figure 2 - Typical combined GRF plot from research papers[2]

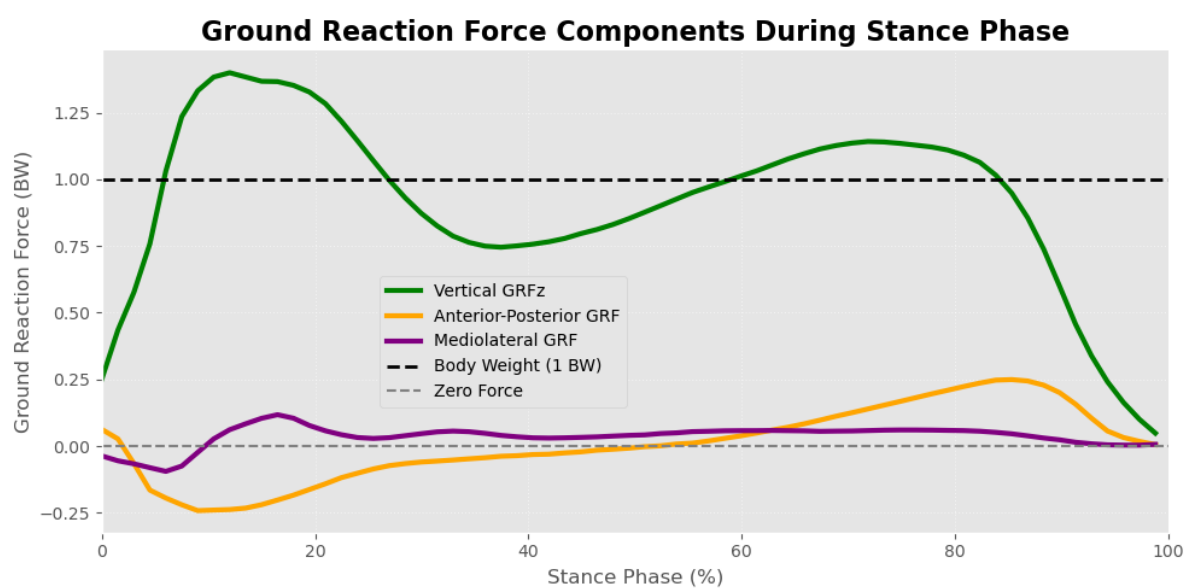


Figure 3 - Combined GRF plot extracted with Python

5.1. GRFz

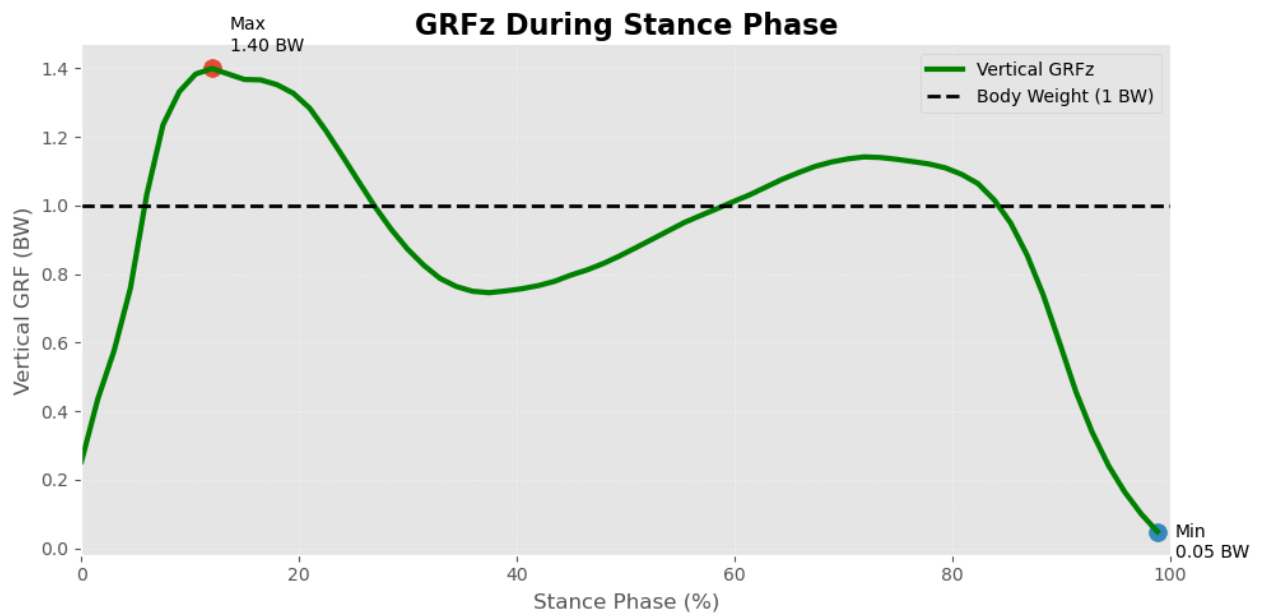


Figure 4 - The vertical GRF exhibited the characteristic double-peaked pattern commonly reported during the stance phase from heel strike to toe off. (0% to 60% gait)

Parameter	Theory	Experimental	Comment
First Peak	1.0 - 1.2	~1.40	Exceeds range
Mid-stance minimum	0.75 - 0.85	~0.75	Within range
Second Peak	1.0 - 1.2	~1.14	Within range

The first peak is the loading response and the second peak is the push-off. In many textbook diagrams; usually peak 2 > peak 1, such as in Figure 2. This can depend on various factors. [3]

A larger first peak can indicate:

- More aggressive heel strike
- Higher downward velocity of the CoM at contact
- Stiffer leg during loading
- Faster walking speed

This will be explored further in this project to determine a reason.

5.2. GRFx & GRFy

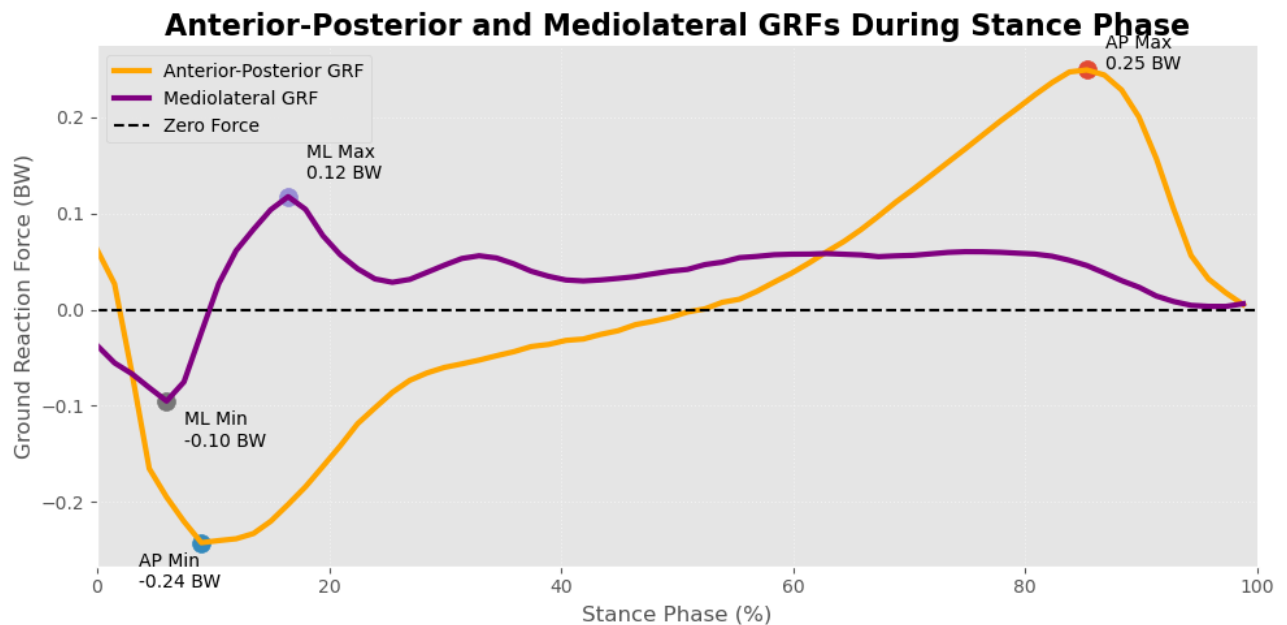


Figure 5 - The anterior-posterior force displayed the expected braking and propulsion phases.

Observed behaviour:

- Negative force during loading response represented braking.
- Positive force during terminal stance represented propulsion.

The braking peak exceeded the propulsion peak, indicating greater deceleration during early stance than acceleration during push-off.

The mediolateral force remained substantially smaller than the vertical force component and reflected normal side-to-side transfer of body mass over the stance limb. [4]

The braking impulse exceeded the propulsive impulse, suggesting the subject experienced greater deceleration during loading response than forward acceleration during push-off. This asymmetry is commonly observed during comfortable walking and does not necessarily indicate an abnormal gait pattern. [4,5]

6. Joint Kinematics

6.1 Hip

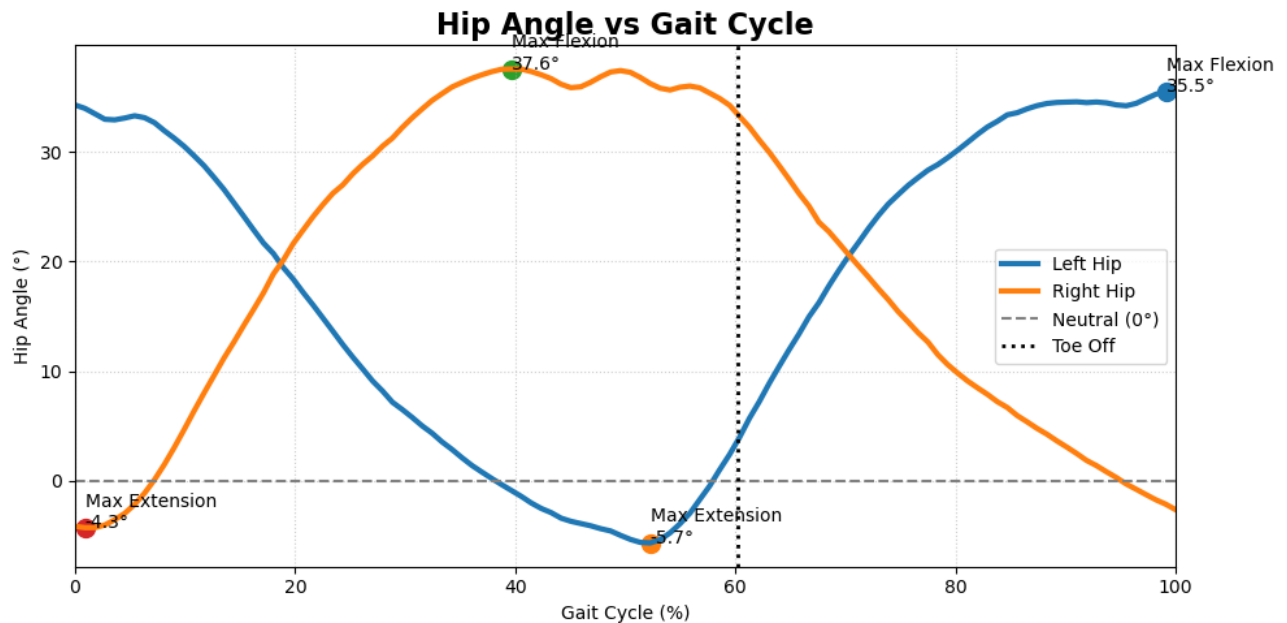


Figure 6- Hip Joint Angles

The hip angle profile closely followed the characteristic pattern reported in the literature, with maximum flexion occurring during late swing/initial contact and maximum extension during terminal stance. [2,3]

Parameter	Theory	Experimental	Comment
Maximum flexion	+25 to +35°	+37.6° (L)	Slightly greater than typical
		+35.5° (R)	
Maximum extension	-10 to -20°	-5.7° (L)	Less extension than expected
		-4.3° (R)	
ROM	+40 to +60°	~42° (L)	On lower end of range
		~41° (R)	

Summary

- Overall waveform closely matched published gait data.
- Hip flexion slightly exceeded literature values.
- Hip extension remained lower than expected.
- Reduced extension is likely multifactorial (short stride, pelvic posture and hip flexor tightness).
- Findings reproduced those obtained in the original assignment.

6.2 Knee

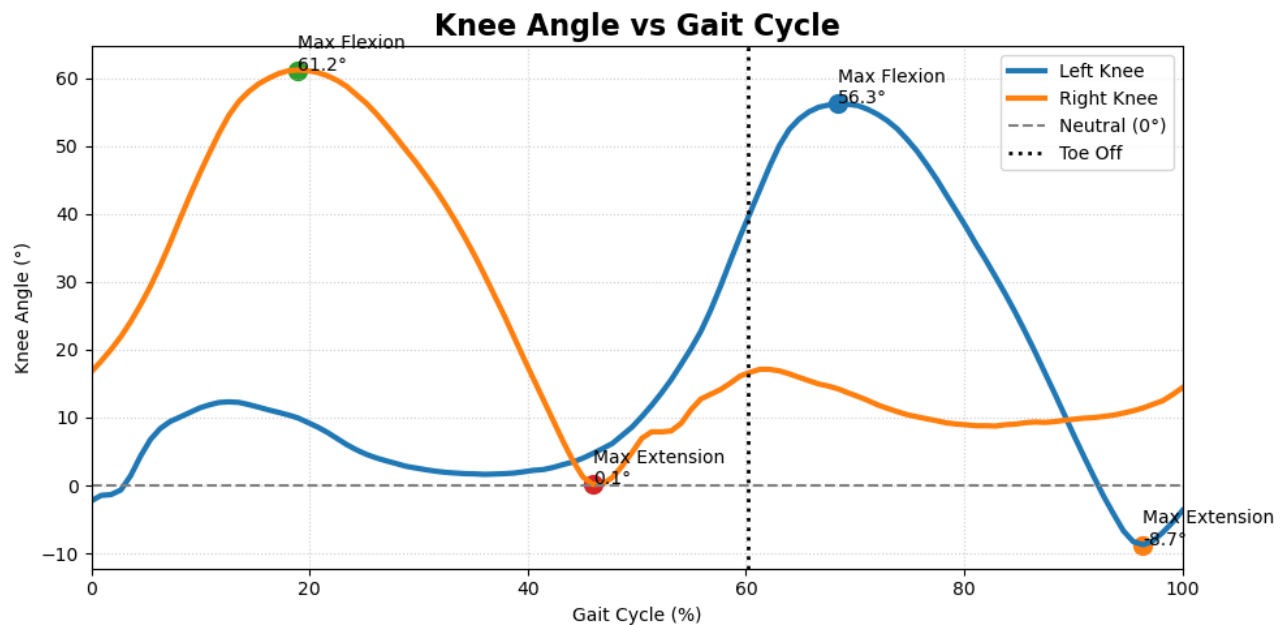


Figure 7 - Knee Joint Angles

The knee angle profile demonstrated the expected loading response flexion, mid-stance extension and swing-phase flexion characteristic of normal walking. The timing of the peaks closely agreed with published gait curves. [2,3]

Parameter	Theory	Experimental	Comment
Initial flexion	+10° to +20°	~12° (L)	Within range
		~17.5° (R)	
Peak swing flexion	+55° to +65°	+56.3° (L)	Within range
		+61.2° (R)	
Terminal swing extension	+0° to +5°	-8.7° (L)	R within range, L slightly greater extension than typical
		+0.1° (R)	
ROM	+55 to +70°	~65° (L)	Within range
		~61° (R)	

Summary

- Excellent agreement with literature.
- Loading response flexion normal.
- Swing-phase flexion normal.
- Mild terminal hyperextension on the left side.
- Overall knee kinematics were successfully reproduced using Python.

6.3 Ankle

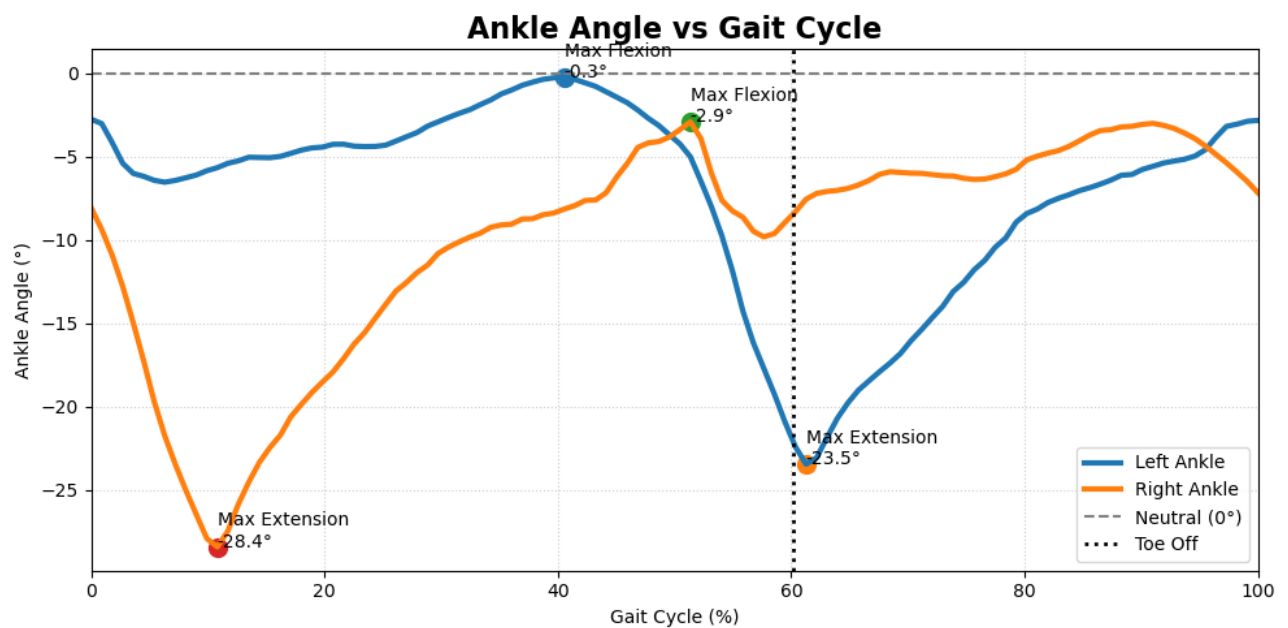


Figure 8 - Ankle Joint Angles

The overall ankle waveform resembled the theoretical gait pattern, demonstrating plantarflexion following heel strike, progressive dorsiflexion during stance and plantarflexion during push-off. However, one notable finding was that dorsiflexion never became positive throughout the gait cycle, unlike the theoretical reference. [2,3]

Parameter	Theory	Experimental	Comment
Maximum dorsiflexion	+10 to +15°	-0.3° (L)	Dorsiflexion was theoretically never reached
		-2.9° (R)	
Maximum plantarflexion	-15 to -25°	-23.5° (L)	Left side on upper range, Right side out of range
		-28.4° (R)	
ROM	+25 to +35°	~23.2° (L)	Left side out of range, Right side barely within range.
		~25.5° (R)	

Summary

- Overall waveform consistent with normal gait.
- Reduced dorsiflexion throughout stance.
- Plantarflexion within expected limits.
- Possible influence of sprinting history and reduced hip extension.
- Reduced ankle mobility rather than abnormal timing.

7. Joint Moments

7.1 Hip

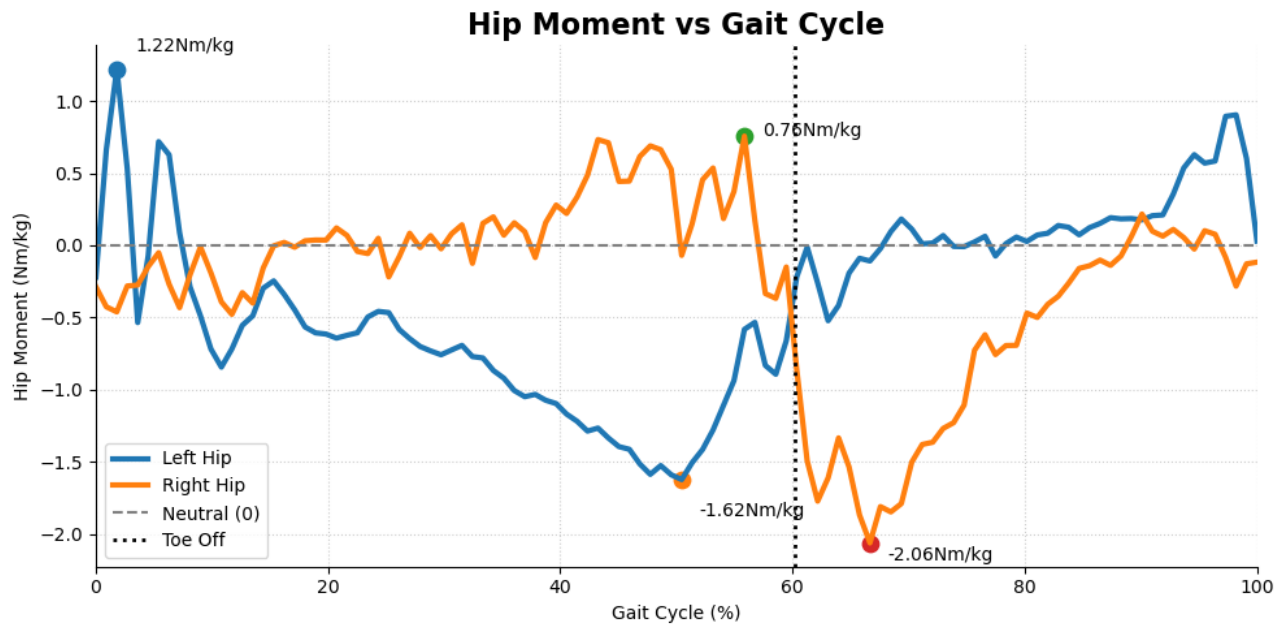


Figure 9. Hip Joint Moments*

Hip moments reflected the muscular demand required to control trunk and lower limb motion throughout stance and swing.

Parameter	Theory (Nm/kg)	Experimental (Nm/kg)	Comment
Peak extension moment	0.8–1.2	1.22 (L)	L just above range, right almost within range
		0.76 (R)	
Peak flexion moment	0.4–0.8	1.62 (L)	Both sides significantly out of range
		2.06 (R)	
Overall pattern	Extension → Flexion	Similar	Agrees with theory

*In graph; extension in +ve direction, flexion in -ve direction

Summary

The hip moment profile demonstrated the expected transition between flexion and extension moments throughout stance and swing. Although peak magnitudes differed somewhat from theoretical values, the overall waveform and timing of moment reversals closely followed published gait patterns. Differences in peak magnitude likely reflect subject-specific gait characteristics, walking speed, stride modifications and the assumptions inherent to inverse dynamics calculations rather than abnormal joint mechanics.

7.2 Knee

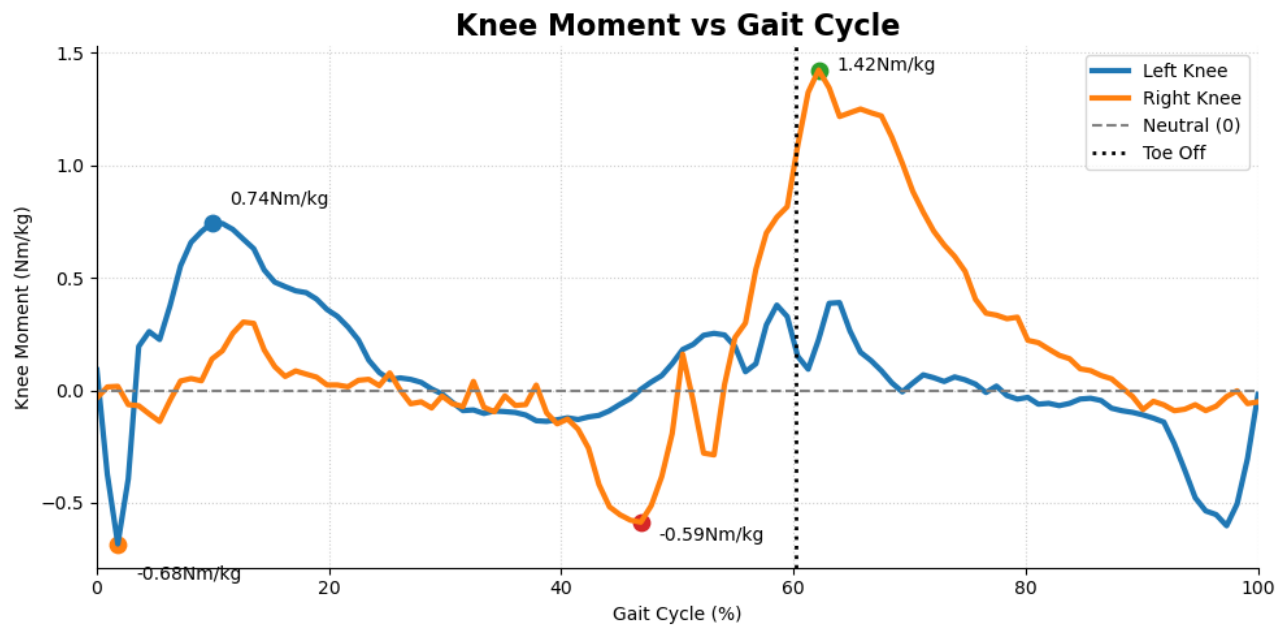


Figure 10 - Knee Joint Moments*

The knee moment profile generally agreed with theoretical gait mechanics, demonstrating an extension moment during early stance followed by a flexion moment later in stance.

Parameter	Theory (Nm/kg)	Experimental (Nm/kg)	Comment
Peak extension moment	0.3–0.6	0.74 (L)	Both sides above range, right side significantly
		1.42 (R)	
Peak flexion moment	0.2–0.5	0.68 (L)	Both sides above range
		0.59 (R)	
Overall pattern	Flexion → Extension → Flexion	Similar	Agrees with theory

*In graph; extension in +ve direction, flexion in -ve direction

Summary

Peak extension moments were greater than theoretical values, which may be related to the slight terminal knee hyperextension observed kinematically or subject-specific loading patterns. However, because joint moments are highly sensitive to inverse dynamics modelling, marker placement and anthropometric assumptions, greater emphasis was placed on the overall waveform rather than absolute peak magnitudes.

7.3 Ankle

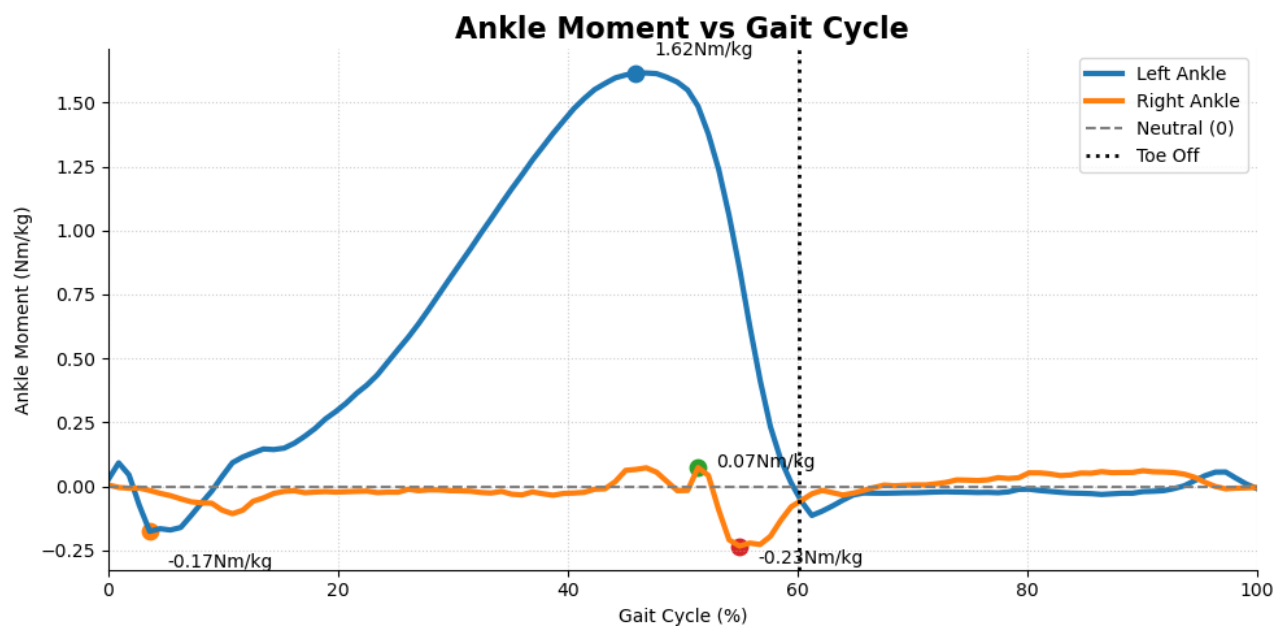


Figure 11 - Ankle Joint Moments*

The ankle moment exhibited the characteristic rapid increase in plantarflexor moment during late stance before returning towards zero after toe-off.

It should be noted that the right ankle moment data contained measurement inconsistencies and should therefore be interpreted with caution.

Parameter	Theory (Nm/kg)	Experimental (Nm/kg)	Comment
Peak plantarflexion moment	1.2–1.8	1.62 (L)	Left within range, Right out of range
		0.07 (R)	
Peak dorsiflexion moment	0.1–0.3	0.17 (L)	Both sides within range
		0.23 (R)	
Overall pattern	Small dorsiflexion → Large plantarflexion	No	Left side in agreement, right side seems corrupted

*In graph; extension in +ve direction, flexion in -ve direction

Summary

The large plantarflexor moment observed is consistent with the substantial contribution of the gastrocnemius and soleus during push-off. Considering the participant's sprinting background, greater plantarflexor strength and ankle stiffness may partially explain the increased peak moment.

8. Pelvic Vertical Oscillation and Walking Speed

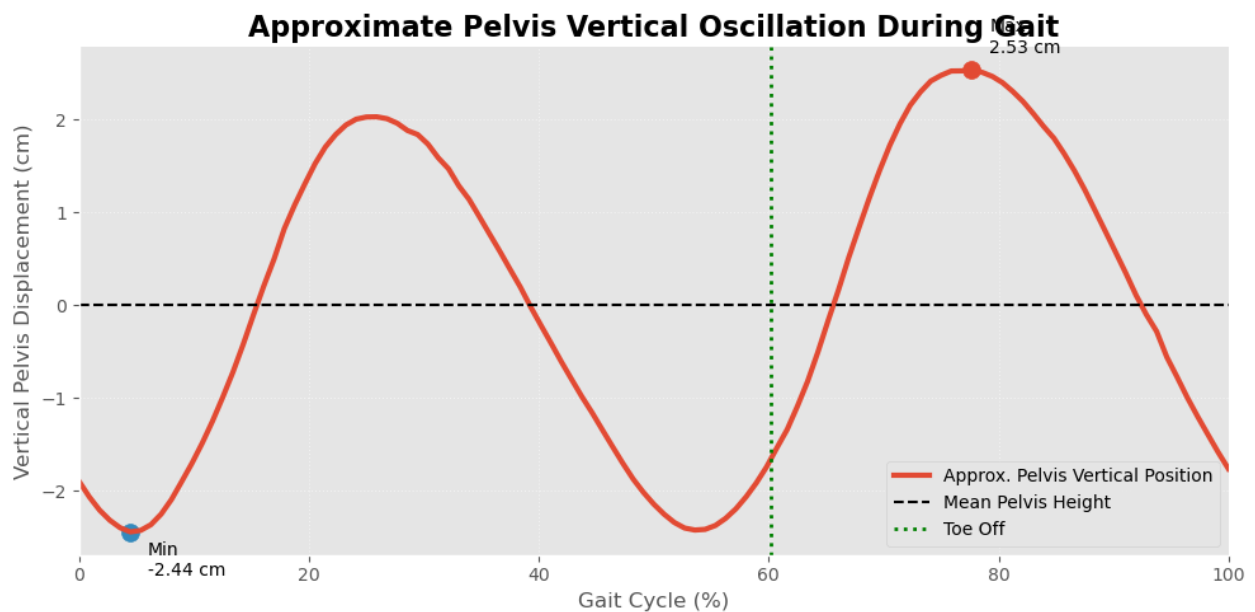


Figure 12 - Pelvic Vertical Oscillation

Pelvic vertical displacement was estimated by averaging the LASI, RASI, LPSI and RPSI marker positions with respect to the max / min positions of the markers:

Results:

Parameter	Theory	Experimental
Vertical pelvis excursion	3-5cm	~4.9 cm

Typical healthy walking is associated with approximately 3–5 cm of vertical pelvic or centre-of-mass excursion. The observed value therefore lies within the normal range.

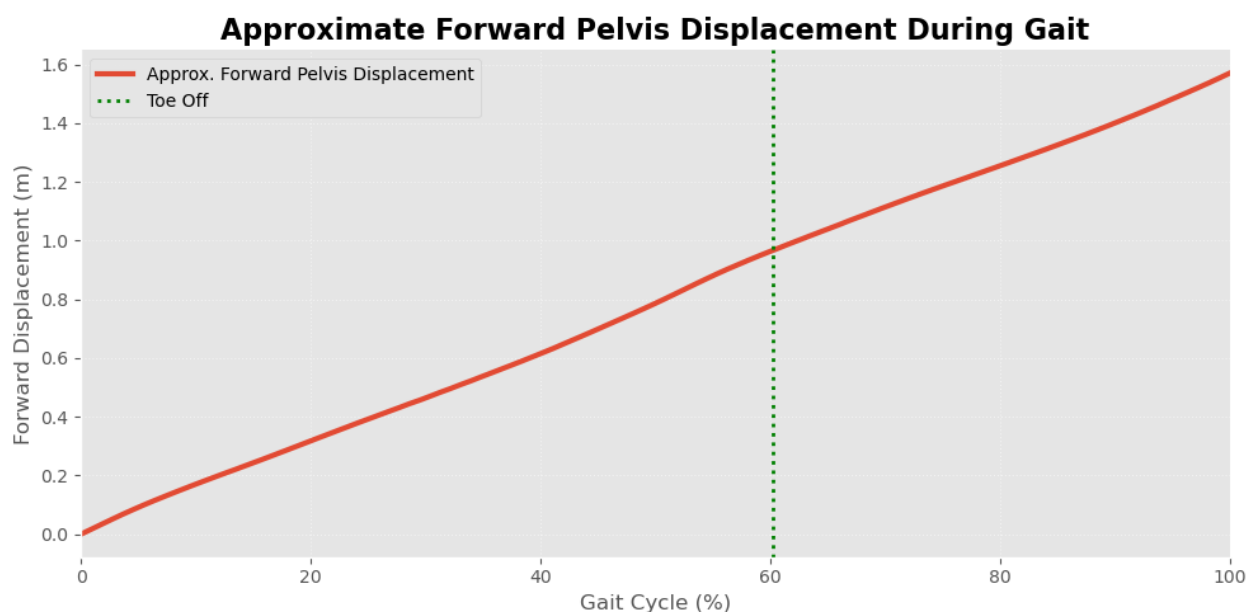


Figure 13 - Forward Pelvis Displacement

Average walking speed was estimated from the forward displacement of the pelvis.

Results:

Parameter	Theory	Experimental
Walking speed	1.0 to 1.5m/s	~1.4 m/s

This value corresponds to normal adult walking speed.

9. Discussion

Overall, the reproduced ground reaction force, kinematic and kinetic waveforms closely matched published healthy gait patterns, demonstrating that the Python workflow successfully reproduced the original biomechanical analysis [1]. The increased first vertical GRF peak (~1.40 BW) most likely reflects a relatively forceful loading response associated with increased lower-limb stiffness during initial contact. Given the participant's sprinting background, this represents a plausible explanation. However, the measured walking speed (~1.4 m/s) and pelvic vertical excursion (~4.9 cm) remained within healthy adult ranges, indicating that the elevated loading response was not caused by excessive vertical body motion or a 'bouncy' gait.

Reduced hip extension was consistent with the findings of the original assignment and is likely multifactorial, arising from the shortened stride adopted to ensure force plate contact, mild anterior pelvic tilt and relatively tight hip flexor musculature. Reduced hip extension may also limit forward tibial progression during terminal stance, partially explaining the reduced ankle dorsiflexion observed later in the gait cycle.

The knee demonstrated the closest agreement with published gait data. Mild terminal hyperextension of the left knee is considered more likely to reflect individual gait characteristics, stride modification or modelling inaccuracies than pathological movement.

Although ankle dorsiflexion remained below theoretical values, the timing of the ankle waveform remained physiological, suggesting reduced ankle mobility rather than altered gait mechanics. Possible contributing factors include limited hip extension, increased ankle stiffness, tight gastrocnemius-soleus musculature, pes cavus and long-term sprint training.

Several peak joint moments exceeded published reference values. Because inverse dynamics calculations are highly sensitive to marker placement, anthropometric assumptions and filtering procedures, greater emphasis was placed on the overall waveform and timing than on absolute peak magnitude.

10. Limitations

- The analysis was based on a single gait cycle and therefore may not fully represent the subject's habitual gait pattern.
- Small errors in marker placement, soft tissue movement and biomechanical model calibration may have influenced calculated joint angles and moments.
- Joint moments obtained through inverse dynamics are sensitive to anthropometric assumptions and force plate measurements.
- The right ankle moment contained greater variability than the remaining kinetic data and should therefore be interpreted with caution.
- Pelvic marker trajectories were used as an approximation of centre-of-mass motion rather than a direct measurement.

11 Future Work

- Automatic gait-event detection.
- Centre of pressure trajectory analysis.
- Joint power calculations.
- Multi-cycle ensemble averaging.
- Comparison with normative gait databases.
- Interactive reporting dashboards.

12. Conclusion

Python successfully reproduced the original biomechanical gait analysis while automating data extraction, normalisation, visualisation and quantitative analysis. In addition to reproducing the original findings, the workflow calculated walking speed and pelvic vertical excursion, extending the original investigation. The participant demonstrated normal cadence, walking speed, stance-to-swing ratio and pelvic vertical excursion. Although increased loading response, reduced hip extension and reduced ankle dorsiflexion were identified, these findings are most consistent with subject-specific biomechanics, sprinting background and experimental constraints rather than pathological gait.

13. References

- [1] Borg Saywell E. Biomechanical Gait Analysis using Motion Capture and Force Plates. University of Malta. Unpublished coursework.
- [2] Whittle MW. Gait Analysis: An Introduction. 5th ed. Elsevier; 2014.
- [3] Perry J, Burnfield JM. Gait Analysis: Normal and Pathological Function. 2nd ed. SLACK Incorporated; 2010.
- [4] Winter DA. Biomechanics and Motor Control of Human Movement. 4th ed. John Wiley & Sons; 2009.
- [5] Neumann DA. Kinesiology of the Musculoskeletal System. 3rd ed. Elsevier; 2017.

Appendix A – Python Source Code

Selected final Python scripts used to reproduce and extend the gait analysis. Full scripts are provided rather than screenshots so that the code remains readable and reusable.

A.1 Software Environment

Software / Library	Purpose
Python	Main programming language
Pandas	Excel import and data handling
Matplotlib	Plot generation
OpenPyXL	Excel reading engine used by Pandas
Visual Studio Code	Code editor / development environment

A.2 Example Processing Snippet

Short example of the general workflow used to read Excel data, normalise time, normalise force to body weight and plot a graph.

```
import pandas as pd
import matplotlib.pyplot as plt

# Example of the processing logic used throughout the project.
df = pd.read_excel("Walk_002.xlsx", header=None)

time = pd.to_numeric(df.iloc[126:238, 0], errors="coerce")
force_z = pd.to_numeric(df.iloc[126:238, 6], errors="coerce")

time_norm = ((time - time.iloc[0]) / (time.iloc[-1] - time.iloc[0])) * 100

body_mass = 78
body_weight = body_mass * 9.81
force_bw_z = force_z / body_weight

plt.plot(time_norm, force_bw_z)
plt.xlabel("Gait Cycle (%)")
plt.ylabel("Vertical GRF (BW)")
plt.show()
```

A.3 Ground Reaction Force Analysis

Script used to generate the vertical GRF, AP/ML GRFs and combined Pedotti-style GRF plots during stance phase.

```
import pandas as pd
import matplotlib.pyplot as plt

# =====
# Metadata
# =====

metadata = {
    "frame_number": 114,
    "first_frame": 336,
    "point_frequency": 100,
    "analog_frequency": 100,
    "bw": 78
}

events = {
    "LHS_start": 3.35,
    "LHS_end": 4.48,
    "RHS": 3.91,
    "LTO": 4.03,
}

# =====
# Read Excel
# =====

df = pd.read_excel("Walk_002.xlsx", header=None)

# =====
# Extract data
# =====

time = pd.to_numeric(df.iloc[126:238, 0], errors="coerce").reset_index(drop=True)
force_x = pd.to_numeric(df.iloc[126:238, 4], errors="coerce").reset_index(drop=True)
force_y = pd.to_numeric(df.iloc[126:238, 5], errors="coerce").reset_index(drop=True)
force_z = pd.to_numeric(df.iloc[126:238, 6], errors="coerce").reset_index(drop=True)

# =====
# Time normalisation to full gait cycle
# =====

time_norm = ((time - time.iloc[0]) / (time.iloc[-1] - time.iloc[0])) * 100

# =====
# Force normalisation to body weight
# =====

body_weight_newtons = metadata["bw"] * 9.81

# Coordinate convention:
# X = mediolateral
# Y = anterior-posterior
# Z = vertical
# Y is inverted so braking is negative and propulsion is positive.

force_bw_ml = -force_x / body_weight_newtons
force_bw_ap = -force_y / body_weight_newtons
force_bw_z = force_z / body_weight_newtons

# =====
# Toe-off position
# =====
```

```

toe_off = (
    (events["LTO"] - events["LHS_start"]) /
    (events["LHS_end"] - events["LHS_start"])
) * 100

# =====
# Crop to stance phase only
# =====

stance_mask = time_norm <= toe_off

time_stance = time_norm[stance_mask].reset_index(drop=True)
force_ml_stance = force_bw_ml[stance_mask].reset_index(drop=True)
force_ap_stance = force_bw_ap[stance_mask].reset_index(drop=True)
force_z_stance = force_bw_z[stance_mask].reset_index(drop=True)

# Normalise stance phase to 0-100%
time_stance_norm = (time_stance / toe_off) * 100

# =====
# Peak / minimum indices
# =====

z_max_idx = force_z_stance.idxmax()
z_min_idx = force_z_stance.idxmin()
ap_max_idx = force_ap_stance.idxmax()
ap_min_idx = force_ap_stance.idxmin()
ml_max_idx = force_ml_stance.idxmax()
ml_min_idx = force_ml_stance.idxmin()

# =====
# Helper function for annotation
# =====

def mark_point(ax, x, y, label, xytext=(10, 10)):
    ax.scatter(x, y, s=80)
    ax.annotate(label, (x, y), xytext=xytext, textcoords="offset points")

plt.style.use("ggplot")

# =====
# Plot 1: Vertical GRFz during stance
# =====

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(time_stance_norm, force_z_stance, linewidth=3, label="Vertical GRFz")
ax.axhline(1, linestyle="--", linewidth=2, color="black", label="Body Weight (1 BW)")

mark_point(
    ax,
    time_stance_norm.iloc[z_max_idx],
    force_z_stance.iloc[z_max_idx],
    f"Max\n{force_z_stance.iloc[z_max_idx]:.2f} BW",
    xytext=(10, 10)
)

mark_point(
    ax,
    time_stance_norm.iloc[z_min_idx],
    force_z_stance.iloc[z_min_idx],
    f"Min\n{force_z_stance.iloc[z_min_idx]:.2f} BW",
    xytext=(10, -25)
)

ax.set_title("GRFz During Stance Phase", fontsize=16, fontweight="bold")
ax.set_xlabel("Stance Phase (%)")
ax.set_ylabel("Vertical GRF (BW)")
ax.set_xlim(0, 100)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend()

```

```

plt.tight_layout()
plt.show()

# =====
# Plot 2: AP and ML GRFs during stance
# =====

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(time_stance_norm, force_ap_stance, linewidth=3, label="Anterior-Posterior GRF")
ax.plot(time_stance_norm, force_ml_stance, linewidth=3, label="Mediolateral GRF")
ax.axhline(0, linestyle="--", linewidth=1.5, color="black", label="Zero Force")

mark_point(ax, time_stance_norm.iloc[ap_min_idx], force_ap_stance.iloc[ap_min_idx],
            f"AP Min\n{force_ap_stance.iloc[ap_min_idx]:.2f} BW", xytext=(10, -25))
mark_point(ax, time_stance_norm.iloc[ap_max_idx], force_ap_stance.iloc[ap_max_idx],
            f"AP Max\n{force_ap_stance.iloc[ap_max_idx]:.2f} BW", xytext=(10, 10))
mark_point(ax, time_stance_norm.iloc[ml_min_idx], force_ml_stance.iloc[ml_min_idx],
            f"ML Min\n{force_ml_stance.iloc[ml_min_idx]:.2f} BW", xytext=(10, -25))
mark_point(ax, time_stance_norm.iloc[ml_max_idx], force_ml_stance.iloc[ml_max_idx],
            f"ML Max\n{force_ml_stance.iloc[ml_max_idx]:.2f} BW", xytext=(10, 10))

ax.set_title("Anterior-Posterior and Mediolateral GRFs During Stance Phase",
             fontsize=16, fontweight="bold")
ax.set_xlabel("Stance Phase (%)")
ax.set_ylabel("Ground Reaction Force (BW)")
ax.set_xlim(0, 100)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend(loc="best")
plt.tight_layout()
plt.show()

# =====
# Plot 3: Combined GRF components
# =====

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(time_stance_norm, force_z_stance, linewidth=3, label="Vertical GRFz")
ax.plot(time_stance_norm, force_ap_stance, linewidth=3, label="Anterior-Posterior GRF")
ax.plot(time_stance_norm, force_ml_stance, linewidth=3, label="Mediolateral GRF")
ax.axhline(1, linestyle="--", linewidth=2, color="black", label="Body Weight (1 BW)")
ax.axhline(0, linestyle="--", linewidth=1.5, color="grey", label="Zero Force")
ax.set_title("Ground Reaction Force Components During Stance Phase", fontsize=16, fontweight="bold")
ax.set_xlabel("Stance Phase (%)")
ax.set_ylabel("Ground Reaction Force (BW)")
ax.set_xlim(0, 100)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend(loc="best")
plt.tight_layout()
plt.show()

# =====
# Metrics
# =====

print(f"Peak GRFz: {force_z_stance.max():.2f} BW")
print(f"Minimum GRFz: {force_z_stance.min():.2f} BW")
print(f"Peak braking force: {force_ap_stance.min():.2f} BW")
print(f"Peak propulsion force: {force_ap_stance.max():.2f} BW")
print(f"Peak mediolateral force: {force_ml_stance.abs().max():.2f} BW")
print(f"Toe-off: {toe_off:.1f}% gait cycle")

```

A.4 Joint Moment Analysis

Script used to plot hip, knee and ankle moments and convert exported Nmm/kg values to Nm/kg.

```
import pandas as pd
import matplotlib.pyplot as plt

# =====
# Metadata
# =====

metadata = {
    "frame_number": 114,
    "first_frame": 336,
    "point_frequency": 100,
    "analog_frequency": 100,
    "bw": 78
}

events = {
    "LHS_start": 3.35,
    "LHS_end": 4.48,
    "RHS": 3.91,
    "LTO": 4.03,
}

# =====
# Read Excel
# =====

df = pd.read_excel("Walk_002.xlsx", header=None)

# =====
# Extract time
# =====

time = pd.to_numeric(df.iloc[125:237, 0], errors="coerce").reset_index(drop=True)

# =====
# Extract joint moments
# =====
# Exported moments are treated as Nmm/kg.
# Dividing by 1000 converts them to Nm/kg.

LHIP_moment = pd.to_numeric(df.iloc[8:120, 82], errors="coerce").reset_index(drop=True) / 1000
LKNEE_moment = pd.to_numeric(df.iloc[8:120, 76], errors="coerce").reset_index(drop=True) / 1000
LANKLE_moment = pd.to_numeric(df.iloc[8:120, 67], errors="coerce").reset_index(drop=True) / 1000

RHIP_moment = pd.to_numeric(df.iloc[8:120, 79], errors="coerce").reset_index(drop=True) / 1000
RKNEE_moment = pd.to_numeric(df.iloc[8:120, 73], errors="coerce").reset_index(drop=True) / 1000
RANKLE_moment = pd.to_numeric(df.iloc[8:120, 70], errors="coerce").reset_index(drop=True) / 1000

# =====
# Time normalisation
# =====

time_norm = ((time - time.iloc[0]) / (time.iloc[-1] - time.iloc[0])) * 100

# =====
# Toe-off position
# =====

toe_off = (
    (events["LTO"] - events["LHS_start"]) /
    (events["LHS_end"] - events["LHS_start"])
) * 100

# =====
```



```

# Plot function
# =====

def plot_moment(time_norm, left, right, joint_name):
    fig, ax = plt.subplots(figsize=(10, 5))

    ax.plot(time_norm, left, linewidth=3, label=f"Left {joint_name}")
    ax.plot(time_norm, right, linewidth=3, label=f"Right {joint_name}")

    ax.axhline(0, linestyle="--", color="grey", linewidth=1.5, label="Neutral (0)")
    ax.axvline(toe_off, linestyle=":", color="black", linewidth=2, label="Toe Off")

    # Left max/min markers
    ax.scatter(time_norm.iloc[left.idxmax()], left.max(), s=80)
    ax.scatter(time_norm.iloc[left.idxmin()], left.min(), s=80)

    # Right max/min markers
    ax.scatter(time_norm.iloc[right.idxmax()], right.max(), s=80)
    ax.scatter(time_norm.iloc[right.idxmin()], right.min(), s=80)

    ax.annotate(
        f"{left.max():.2f} Nm/kg",
        (time_norm.iloc[left.idxmax()], left.max()),
        xytext=(10, 10),
        textcoords="offset points"
    )

    ax.annotate(
        f"{left.min():.2f} Nm/kg",
        (time_norm.iloc[left.idxmin()], left.min()),
        xytext=(10, -20),
        textcoords="offset points"
    )

    ax.annotate(
        f"{right.max():.2f} Nm/kg",
        (time_norm.iloc[right.idxmax()], right.max()),
        xytext=(10, 0),
        textcoords="offset points"
    )

    ax.annotate(
        f"{right.min():.2f} Nm/kg",
        (time_norm.iloc[right.idxmin()], right.min()),
        xytext=(10, -10),
        textcoords="offset points"
    )

    ax.set_title(f"{joint_name} Moment vs Gait Cycle", fontsize=16, fontweight="bold")
    ax.set_xlabel("Gait Cycle (%)")
    ax.set_ylabel(f"{joint_name} Moment (Nm/kg)")
    ax.set_xlim(0, 100)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(True, linestyle=":", alpha=0.6)
    ax.legend()
    plt.tight_layout()
    plt.show()

# =====
# Max-min function
# =====

def print_max_min(left, right, joint_name):
    print(f"\n{joint_name}")
    print("-" * 30)
    print(f"Left Max : {left.max():.2f} Nm/kg")
    print(f"Left Min : {left.min():.2f} Nm/kg")
    print(f"Right Max: {right.max():.2f} Nm/kg")
    print(f"Right Min: {right.min():.2f} Nm/kg")

# =====
# Generate graphs
# =====

```

```
plot_moment(time_norm, LHIP_moment, RHIP_moment, "Hip")
plot_moment(time_norm, LKNEE_moment, RKNEE_moment, "Knee")
plot_moment(time_norm, LANKLE_moment, RANKLE_moment, "Ankle")
```

```
# =====
# Print metrics
# =====
```

```
print_max_min(LHIP_moment, RHIP_moment, "Hip")
print_max_min(LKNEE_moment, RKNEE_moment, "Knee")
print_max_min(LANKLE_moment, RANKLE_moment, "Ankle")
```

A.5 Pelvic Vertical Oscillation and Walking Speed

Script used to estimate pelvic vertical excursion and forward walking speed from pelvis marker trajectories.

```
import pandas as pd
import matplotlib.pyplot as plt

# =====
# Metadata / Events
# =====

events = {
    "LHS_start": 3.35,
    "LHS_end": 4.48,
    "LTO": 4.03,
}

# =====
# Read TRC Excel file
# =====

df = pd.read_excel("Walk_002_markers.TRC.xlsx", header=None)

# =====
# Extract data rows
# =====
# The TRC file contains headers at the top; the gait cycle starts at frame 336.

time = pd.to_numeric(df.iloc[82:195, 1], errors="coerce").reset_index(drop=True)

# =====
# Extract pelvis marker vertical coordinates
# =====
# In the exported Excel file, the vertical motion corresponded to the X columns.
# LASI = columns 2,3,4
# RASI = columns 5,6,7
# LPSI = columns 8,9,10
# RPSI = columns 11,12,13

LASI_vertical = pd.to_numeric(df.iloc[82:195, 2], errors="coerce").reset_index(drop=True)
RASI_vertical = pd.to_numeric(df.iloc[82:195, 5], errors="coerce").reset_index(drop=True)
LPSI_vertical = pd.to_numeric(df.iloc[82:195, 8], errors="coerce").reset_index(drop=True)
RPSI_vertical = pd.to_numeric(df.iloc[82:195, 11], errors="coerce").reset_index(drop=True)

# Forward displacement was taken from the coordinate that increased steadily during gait.
# Update this column if the coordinate convention differs in another export.
LASI_forward = pd.to_numeric(df.iloc[82:195, 4], errors="coerce").reset_index(drop=True)
RASI_forward = pd.to_numeric(df.iloc[82:195, 7], errors="coerce").reset_index(drop=True)
LPSI_forward = pd.to_numeric(df.iloc[82:195, 10], errors="coerce").reset_index(drop=True)
RPSI_forward = pd.to_numeric(df.iloc[82:195, 13], errors="coerce").reset_index(drop=True)

# =====
# Approximate pelvis centre
# =====

pelvis_vertical_mm = (LASI_vertical + RASI_vertical + LPSI_vertical + RPSI_vertical) / 4
pelvis_forward_mm = (LASI_forward + RASI_forward + LPSI_forward + RPSI_forward) / 4

# Convert mm to cm/metres
pelvis_vertical_cm = pelvis_vertical_mm / 10
pelvis_forward_m = pelvis_forward_mm / 1000

# =====
# Crop to one gait cycle
# =====

gait_mask = (time >= events["LHS_start"]) & (time <= events["LHS_end"])
```

```

time_gait = time[gait_mask].reset_index(drop=True)
pelvis_vertical_gait = pelvis_vertical_cm[gait_mask].reset_index(drop=True)
pelvis_forward_gait = pelvis_forward_m[gait_mask].reset_index(drop=True)

# =====
# Normalise gait cycle to 0-100%
# =====

time_norm = ((time_gait - time_gait.iloc[0]) / (time_gait.iloc[-1] - time_gait.iloc[0])) * 100

# =====
# Relative displacement
# =====

pelvis_vertical_relative = pelvis_vertical_gait - pelvis_vertical_gait.mean()
pelvis_forward_relative = pelvis_forward_gait - pelvis_forward_gait.iloc[0]

# =====
# Toe-off position
# =====

toe_off = (
    (events["LTO"] - events["LHS_start"]) /
    (events["LHS_end"] - events["LHS_start"])
) * 100

# =====
# Metrics
# =====

vertical_excursion = pelvis_vertical_relative.max() - pelvis_vertical_relative.min()
forward_distance = pelvis_forward_relative.iloc[-1] - pelvis_forward_relative.iloc[0]
gait_cycle_time = events["LHS_end"] - events["LHS_start"]
walking_speed = forward_distance / gait_cycle_time

max_idx = pelvis_vertical_relative.idxmax()
min_idx = pelvis_vertical_relative.idxmin()

# =====
# Plot 1: Pelvis vertical oscillation
# =====

plt.style.use("ggplot")

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(time_norm, pelvis_vertical_relative, linewidth=3, label="Approx. Pelvis Vertical Position")
ax.axhline(0, linestyle="--", color="black", linewidth=1.5, label="Mean Pelvis Height")
ax.axvline(toe_off, linestyle=":", color="green", linewidth=2, label="Toe Off")

ax.scatter(time_norm.iloc[max_idx], pelvis_vertical_relative.iloc[max_idx], s=100)
ax.annotate(
    f"Max\n{pelvis_vertical_relative.iloc[max_idx]:.2f} cm",
    (time_norm.iloc[max_idx], pelvis_vertical_relative.iloc[max_idx]),
    xytext=(10, 10),
    textcoords="offset points"
)

ax.scatter(time_norm.iloc[min_idx], pelvis_vertical_relative.iloc[min_idx], s=100)
ax.annotate(
    f"Min\n{pelvis_vertical_relative.iloc[min_idx]:.2f} cm",
    (time_norm.iloc[min_idx], pelvis_vertical_relative.iloc[min_idx]),
    xytext=(10, -25),
    textcoords="offset points"
)

ax.set_title("Approximate Pelvis Vertical Oscillation During Gait", fontsize=16, fontweight="bold")
ax.set_xlabel("Gait Cycle (%)")
ax.set_ylabel("Vertical Pelvis Displacement (cm)")
ax.set_xlim(0, 100)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend()

```

```

plt.tight_layout()
plt.show()

# =====
# Plot 2: Forward pelvis displacement
# =====

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(time_norm, pelvis_forward_relative, linewidth=3, label="Approx. Forward Pelvis Displacement")
ax.axvline(toe_off, linestyle=":", color="green", linewidth=2, label="Toe Off")
ax.set_title("Approximate Forward Pelvis Displacement During Gait", fontsize=16, fontweight="bold")
ax.set_xlabel("Gait Cycle (%)")
ax.set_ylabel("Forward Displacement (m)")
ax.set_xlim(0, 100)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend()
plt.tight_layout()
plt.show()

# =====
# Print metrics
# =====

print(f"Vertical pelvis excursion: {vertical_excursion:.2f} cm")
print(f"Forward pelvis displacement: {forward_distance:.2f} m")
print(f"Gait cycle time: {gait_cycle_time:.2f} s")
print(f"Estimated walking speed: {walking_speed:.2f} m/s")
print(f"Toe-off: {toe_off:.1f}% gait cycle")

```